VeilPay

PRIVACY-FIRST / PASSKEY-SIGNED / PAYPAL SANDBOX

When you pay someone, they never learn your phone or email, and no payment leaves your account unless you personally signed that exact amount to that exact person.

PayPal AI Hackathon 2026
Submission package

Flask 3 / MongoDB Atlas
PayPal Orders + Payments APIs

27 automated tests
zero-PII leak invariants

All payments run in the PayPal Sandbox with synthetic accounts and test currency.

01 / THE PROBLEM

Paying someone shouldn't mean handing them your identity

Two failures sit inside every mainstream payment app. VeilPay closes both with one design.

Contact leakage

On GPay, CashApp and Venmo, the counterparty can often see your phone number, real name or email, and use it to spam, harass or harvest contact lists.

Unconsented transfers

Phishing, session hijacking and SIM-swap or OTP interception let money leave an account without the owner approving that exact transaction.

Zero-PII transactions: users transact only via aliases, and every transfer is cryptographically bound to a passkey signature, with a "This was not me" safety hold.

Five ideas, one product

1. Alias-first privacy

Every user gets an unforgeable, server-generated alias such as `veil-7x3k9m`. Payer and payee see only aliases. Allowlist serializers ensure no API response, log or view can leak a real email or phone number.

2. Passkey-bound payments

WebAuthn passkeys replace passwords and OTPs. Each payment intent is canonically hashed with SHA-256, and the WebAuthn challenge is exactly that hash. Change the amount or payee by one cent and the signature fails.

3. Safety hold and void

After PayPal authorization, funds sit in `AUTHORIZED_HOLD` for 60 to 300 seconds. One tap on "This was not me" voids the authorization before capture.

4. AI risk and privacy guardian

Deterministic rules decide. An LLM explains the risk in 25 words or fewer and can only raise risk, never lower it. A two-pass scrubber strips phone numbers, emails and UPI handles from notes.

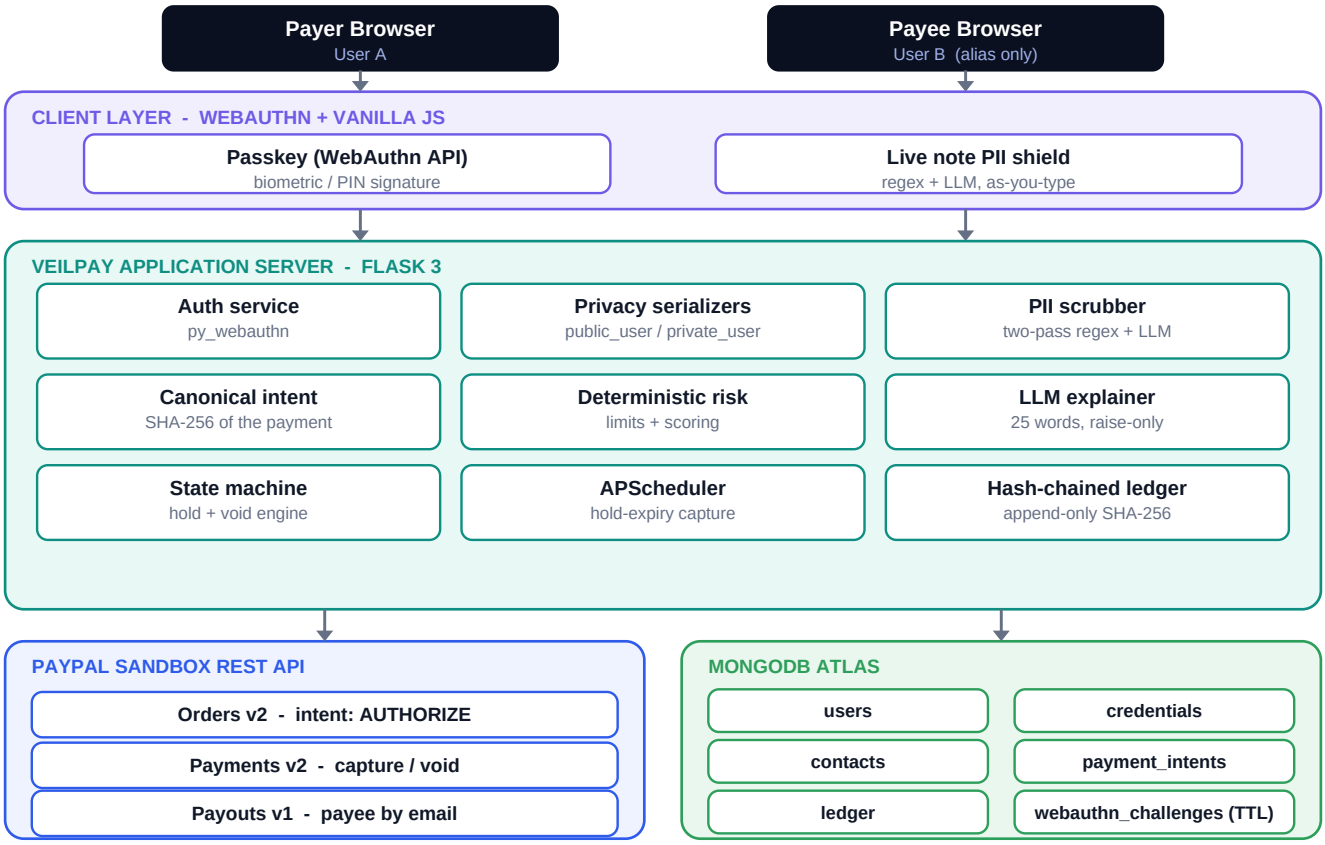
5. Tamper-evident ledger

Every state transition is appended to an append-only ledger, chained with SHA-256, so history cannot be quietly rewritten.

02 / SYSTEM DESIGN

Architecture

A thin browser client, a Flask server that owns every decision, and two external systems: PayPal for money, MongoDB for state.

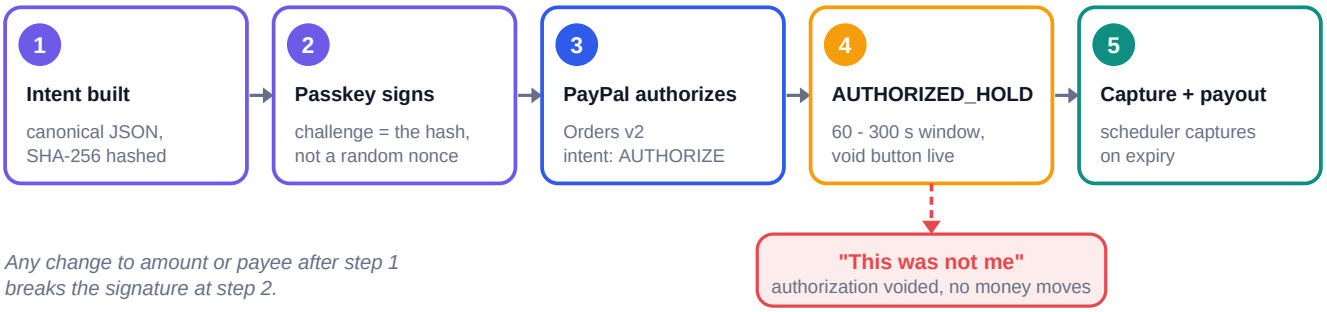


Reading the diagram. The payer's browser registers and logs in through the auth service, then submits a passkey signature to the intent engine. Risk is scored deterministically first; the LLM only adds an explanation or raises the tier. The state machine talks to PayPal Orders and Payments, the scheduler captures or pays out when a hold expires, and every transition lands in the ledger. The payee's browser only ever receives data through the privacy serializers.

03 / PAYMENT LIFECYCLE

From tap to settled, with an exit ramp

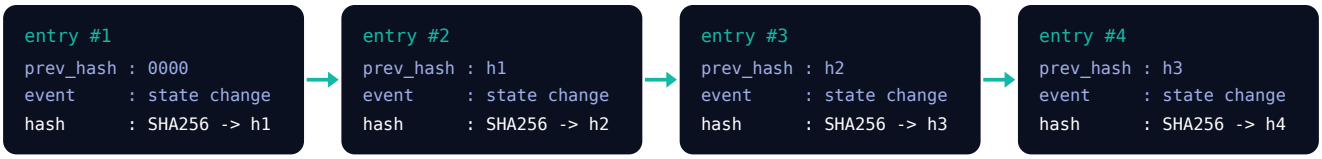
The signed hash is the spine of the flow. Everything after it either matches it exactly or is rejected.



Any change to amount or payee after step 1 breaks the signature at step 2.

The tamper-evident ledger

Each state change appends one entry that includes the hash of the previous entry. Editing or deleting any past record changes every hash after it, which makes tampering visible.



AI guardrails: advisory, never authoritative

Layer	What it does	What it cannot do
Deterministic risk engine	Applies per-payment and per-day caps, scores the payment, and sets the hold length.	Nothing here depends on a model.
LLM explainer	Writes a plain-language reason in 25 words or fewer, e.g. "First payment to this person, so we added a safety hold." May raise the risk tier.	Approve, deny or lower risk. Output is schema-validated JSON.
PII scrubber	Pass 1 regex catches phones, emails and UPI handles. Pass 2 LLM catches obfuscated forms such as spelled-out numbers or "name at gmail dot com". Matches become `[hidden]`.	Trust the note. It is wrapped as untrusted data to blunt prompt injection.

04 / SECURITY

Threat model and mitigations

Each threat is paired with the control that answers it and the risk we openly keep.

Threat	VeilPay mitigation	Residual risk
Stolen session cookie	State-changing payments require a fresh WebAuthn assertion; 30-minute idle session timeout.	Attacker can view alias-only activity.
Phishing / fake domains	Passkeys are bound to the site origin (`RP_ID`), so credentials cannot be given to a look-alike site.	User approves a payment to a legitimate-looking but fraudulent counterparty.
Payee harvesting payer contact info	Strict allowlist serializers (`public_user`); payer details scrubbed and Fernet-encrypted at rest.	Data held on PayPal's platform follows PayPal's privacy policy.
Amount or payee tampering	Challenge is `SHA256(canonical_intent)`; any change breaks signature verification.	None within the signed flow.
Replay attacks	Single-use nonces, consumed-challenge records, 5-minute TTL index on challenges.	None expected.
Prompt injection in notes	LLM output is schema-validated, the memo is wrapped as untrusted data, and the LLM can only raise risk.	Minor oddities in explanation phrasing.
Compromised device	Hold countdown, prominent "This was not me" void, per-payment and per-day caps.	Loss up to the limit if the user ignores every alert.
Platform insider	Platform-only emails encrypted with Fernet; zero secret logging.	Platform holds the keys (documented limitation).

Security Lab: attacks you can run live

The `/demo` panel lets a judge fire three attacks and watch each one fail, with the reason shown on screen.

Attack	Result	Why it fails
Pay without a signature	`BLOCKED_BY_POLICY`	State-changing payments need a fresh passkey assertion.
Replay a signed payment	`BLOCKED_BY_REPLAY_PROTECTION`	Challenge already consumed; nonces are single-use.
Edit the amount in transit	`BLOCKED_BY_CRYPTOGRAPHIC_BINDING`	SHA-256 of the edited intent no longer matches the signed challenge; both hashes are shown.

05 / TRANSPARENCY

What is real, what is simulated, and how AI is used

Judges should know exactly what they are looking at.

REAL

- WebAuthn passkey registration, login and payment signing with `py_webauthn`
- SHA-256 canonical intent binding
- Two-pass regex + LLM PII scrubbing
- Deterministic risk engine
- Append-only, SHA-256 hash-chained ledger
- Fernet encryption of PII at rest
- PayPal Orders v2 (`intent: AUTHORIZE`) and Payments v2 capture / void

SIMULATED / SANDBOX

- Everything runs in the PayPal Sandbox with synthetic test accounts and synthetic USD balances
- If the sandbox Business account lacks Payouts API permission, the platform records a simulated payout with clear audit flags in the ledger

AI disclosure

Models	Groq API (`openai/gpt-oss-120b` / Llama 3.3 70B) and Google Gemini 1.5 Flash via the Gemini API, both returning strict schema-validated JSON.
Role	Explain risk factors in plain language (25 words max), suggest saving repeat payees to the address book, and detect obfuscated PII in payment notes.
Authority	None over money. Deterministic rules govern every decision. AI may add context or raise risk, and can never approve, deny or lower it.

Honest limitations

Limitation	Detail
Privacy between users, not from the platform	The platform encrypts and holds user identity for regulatory compliance.
Regulatory controls	Production needs financial licensing, KYC and AML checks.
Web-level confirmation	The signature binds intent at the browser WebAuthn layer. A native app with a hardware-isolated Trusted Display would guard better against local OS malware.
PayPal Sandbox	Synthetic accounts and test currency throughout.
Fraud mitigation	VeilPay substantially reduces unauthorized payments and contact leakage. It does not claim to eliminate all fraud.

06 / DEMO

Try it in six steps

Open `http://localhost:5000` in Chrome, Edge or Safari to get native WebAuthn biometrics.

Passkey onboarding

Choose Get Passkey. Enter a public display name (e.g. apple, green, Tina) and an email used only for payouts, never shown to counterparties. Tap "Continue with fingerprint or face".

Pay three ways

At /pay: tap a saved contact (or Load Demo Friends for green, apple, Bob), scan or upload a QR from camera or gallery, or just type a nickname or alias.

Sign and hold

Review & Sign shows the canonical intent and its SHA-256. Sign with Passkey, then watch the 60-second hold countdown. Tap "This was not me" to void the PayPal authorization.

Your alias and QR

The Dashboard shows your virtual card with a nickname and permanent `veil-xxxxxx` alias, plus a personal QR card with Copy Payment Link and Save QR Image.

Live privacy shield

In the memo, type: *Call me at 9876543210 or email me at test at gmail dot com*. Watch both contact details become `[hidden]` before authorization.

Security Lab

At /demo, run the three attacks and read the verdicts and mismatched hashes.

Two-and-a-half-minute video script

0:00	**The problem**	Standard apps leak phone numbers, names and emails, enabling harvesting and spam.
0:20	**Passkey registration**	No passwords, no SMS OTPs, hardware-backed WebAuthn.
0:35	**Your alias**	Dashboard with alias and personal QR.
0:50	**Send a payment**	Review screen with the cryptographic canonical summary.
1:10	**AI layer and scrubber**	Phone number masked live; risk badge with plain-language reason.
1:35	**Hold and void**	60-second AUTHORIZED_HOLD, then "This was not me".
1:55	**Attack simulation**	All three attacks cryptographically blocked.
2:15	**Payee view**	"Received from veil-xxxx" with zero contact leakage.
2:25	**Close**	*VeilPay: when you pay someone, they never learn your phone or email, and no money leaves your account without your cryptographic signature.*

07 / RUN IT YOURSELF

Setup

Python 3.11+ (tested on 3.12.10), MongoDB Atlas (free M0) or local MongoDB, a PayPal Developer account, and a Groq or Gemini API key.

PayPal sandbox accounts needed

Account	Purpose
Business	Platform application credentials (`PAYPAL_CLIENT_ID`, `PAYPAL_CLIENT_SECRET`).
Personal (payer)	Authorizes sandbox orders.
Personal (payee)	Receives synthetic payouts.

Install and run

```
git clone <repo-url> && cd veilpay
python3 -m venv .venv && source .venv/bin/activate
#  Windows PowerShell:  .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
cp .env.example .env        # then fill in values below
pytest -v                   # expect 27 passed
python run.py               # open http://localhost:5000
```

`.env` configuration

```
FLASK_SECRET_KEY=your-secure-random-secret
MONGODB_URI=mongodb+srv://<user>:<password>@<cluster>.mongodb.net/veilpay
RP_ID=localhost
RP_NAME=VeilPay
ORIGIN=http://localhost:5000
PAYPAL_CLIENT_ID=your_sandbox_client_id
PAYPAL_CLIENT_SECRET=your_sandbox_client_secret
PAYPAL_BASE_URL=https://api-m.sandbox.paypal.com
FERNET_KEY=your_generated_fernet_key
LLM_API_KEY=your_groq_or_gemini_api_key
LLM_MODEL=openai/gpt-oss-120b
DEMO_MODE=true
LOW_HOLD_SECONDS=60
MEDIUM_HOLD_SECONDS=60
INTENT_TTL_SECONDS=300
```

What the 27 tests cover: zero-PII leak invariants, WebAuthn assertion verification, anti-replay challenge nonces, hold and void state transitions, and the deterministic risk rules.

> **VeilPay: when you pay someone, they never learn your phone or email, and no money leaves your account without your cryptographic signature.**